МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЗАПОРІЗЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ

МАТЕМАТИЧНИЙ ФАКУЛЬТЕТ

КАФЕДРА ПРОГРАМНОЇ ІНЖЕНЕРІЇ

Звіт з лабораторної роботи № 3

з «Конструювання програмного забезпечення»

Виконав: студент групи 6.1214-пі2

Крівокоритов Олександр

Перевірив: Мильцев Олександр Михайлович

Запоріжжя 2026

Код

```
public class Vehicle {
   private final String owner, brand, model, color;
   private final int year;
   private double x, y, direction, speed, fuel;

   public Vehicle(String owner, String brand, String model, int year, String
color) {
      this.owner = (owner == null || owner.isBlank()) ? "Unknown" : owner;
      this.brand = brand; this.model = model;
      this.year  = year;  this.color = color;
   }
   public Vehicle(String owner, String brand, String model) { this(owner, brand,
model, 2020, "White"); }

   public String getOwner()     { return owner; }
   public String getBrand()     { return brand; }
   public String getModel()     { return model; }
   public int    getYear()      { return year;  }
   public String getColor()     { return color; }
   public double getX()         { return x; }
   public double getY()         { return y; }
   public double getDirection() { return direction; }
   public double getSpeed()     { return speed; }
   public double getFuel()      { return fuel; }

   public void setDirection(double d) { direction = ((d % 360) + 360) % 360; }
   public void setSpeed(double s) {
      if (s < 0) System.out.println("[ERROR] Speed cannot be negative!");
      else speed = s;
   }

   public void addFuel(double amount) {
      if (amount <= 0) { System.out.println("[ERROR] Fuel must be positive!");
return; }
      fuel += amount;
      System.out.printf("  +%.1f L | Tank: %.1f L%n", amount, fuel);
   }
```

```
    public void drive(double hours) {
        if (speed <= 0) { System.out.println("[ERROR] Set speed > 0 first!");
return; }
        double dist = speed * hours, need = dist * 0.08;
        if (need > fuel) { System.out.println("[ERROR] Not enough fuel! Need: " +
need + " L"); return; }
        x += dist * Math.cos(Math.toRadians(direction));
        y += dist * Math.sin(Math.toRadians(direction));
        fuel -= need;
        System.out.printf("  %.1f km | (%.2f, %.2f) | Tank: %.2f L%n", dist, x, y,
fuel);
    }

    public void turn(double deg) {
        setDirection(direction + deg);
        System.out.printf("  Turn %.0f deg | Direction: %.0f deg%n", deg,
direction);
    }

    public void stop() { speed = 0; System.out.println("  Stopped."); }

    public double distanceFromOrigin() { return Math.sqrt(x * x + y * y); }

    @Override
    public String toString() {
        return String.format("%d %s %s %s | Owner: %s | (%.2f,%.2f) | %.0f deg |
%.1f km/h | %.2f L",
                year, color, brand, model, owner, x, y, direction, speed, fuel);
    }
}


public class Polynom {
    private int n;
    private double[] a;

    void set(double[] a) {
        if (a == null || a.length == 0) { this.n = 1; this.a = new double[]{0}; return;
}
```

```
        this.n = a.length; this.a = new double[n];
        for (int i = 0; i < n; i++) this.a[i] = a[i];
    }
    void set(int n, double z) { this.n = n; this.a = new double[n]; for (int i = 0; i <
n; i++) this.a[i] = z; }
    void set(int n) { set(n, 0); }

    Polynom(double[] a) { set(a); }
    Polynom(int n, double z) { set(n, z); }
    Polynom(int n) { set(n); }

    double value(double x) {
        double z = 0, q = 1;
        for (int i = 0; i < n; i++) { z += a[i] * q; q *= x; }
        return z;
    }

    void show() {
        System.out.print("Degree of x:\t");
        for (int i = 0; i < n - 1; i++) System.out.print(" " + i + "\t");
        System.out.println(" " + (n - 1));
        System.out.print("Coefficient:\t");
        for (int i = 0; i < n - 1; i++) System.out.print(a[i] + "\t");
        System.out.println(a[n - 1]);
    }
    void show(double x) { System.out.println("x=" + x + " | P(x)=" + value(x)); }

    Polynom diff() {
        Polynom t = new Polynom(n - 1);
        for (int i = 0; i < n - 1; i++) t.a[i] = a[i + 1] * (i + 1);
        return t;
    }
    Polynom diff(int k) {
        if (k >= n) return new Polynom(1);
        return k > 0 ? diff().diff(k - 1) : new Polynom(a);
    }

    Polynom plus(Polynom Q) {
        Polynom t = n >= Q.n ? new Polynom(a) : new Polynom(Q.a);
```

```
        Polynom s = n >= Q.n ? Q : this;
        for (int i = 0; i < s.n; i++) t.a[i] += s.a[i];
        return t;
    }
    Polynom minus(Polynom Q)  { return plus(Q.prod(-1)); }
    Polynom div(double z)     { return prod(1.0 / z); }
    Polynom prod(double z)    { Polynom t = new Polynom(a); for (int i = 0; i <
t.n; i++) t.a[i] *= z; return t; }
    Polynom prod(Polynom Q) {
        Polynom t = new Polynom(n + Q.n - 1);
        for (int i = 0; i < n; i++)
            for (int j = 0; j < Q.n; j++) t.a[i + j] += a[i] * Q.a[j];
        return t;
    }

    // Task 3: static versions
    static Polynom   add(Polynom P, Polynom Q)       { return P.plus(Q); }
    static Polynom   sub(Polynom P, Polynom Q)       { return P.minus(Q); }
    static Polynom   multiply(Polynom P, Polynom Q)  { return P.prod(Q); }
    static Polynom   derivative(Polynom P, int k)    { return P.diff(k); }
    static double    evaluate(Polynom P, double x)   { return P.value(x); }

    // Task 4: new static methods
    static Polynom integrate(Polynom P) {
        Polynom t = new Polynom(P.n + 1);
        for (int i = 0; i < P.n; i++) t.a[i + 1] = P.a[i] / (i + 1);
        return t;
    }
    static double definiteIntegral(Polynom P, double from, double to) {
        Polynom I = integrate(P); return I.value(to) - I.value(from);
    }
    static double norm(Polynom P) {
        double s = 0; for (double c : P.a) s += c * c; return Math.sqrt(s);
    }
    static Polynom fromRoots(double[] roots) {
        Polynom r = new Polynom(new double[]{1.0});
        for (double root : roots) r = r.prod(new Polynom(new double[]{-root,
1.0}));
        return r;
```

```
    }
}


public class MyMath {
    static double L = Math.PI;

    static double Exp(double x, int N) {
        double s = 0, q = 1;
        for (int i = 0; i < N; i++) { s += q; q *= x / (i + 1); }
        return s + q;
    }
    static double Sin(double x, int N) {
        double s = 0, q = x;
        for (int i = 0; i < N; i++) { s += q; q *= (-1) * x * x / (2*i+2) / (2*i+3); }
        return s + q;
    }
    static double Cos(double x, int N) {
        double s = 0, q = 1;
        for (int i = 0; i < N; i++) { s += q; q *= (-1) * x * x / (2*i+1) / (2*i+2); }
        return s + q;
    }
    static double BesselJ(double x, int N) {
        double s = 0, q = 1;
        for (int i = 0; i < N; i++) { s += q; q *= (-1) * x * x / 4 / (i+1) / (i+1); }
        return s + q;
    }
    static double FourSin(double x, double[] a) {
        double s = 0;
        for (int i = 0; i < a.length; i++) s += a[i] * Math.sin(Math.PI * x * (i+1) /
L);
        return s;
    }
    static double FourCos(double x, double[] a) {
        double s = 0;
        for (int i = 0; i < a.length; i++) s += a[i] * Math.cos(Math.PI * x * i / L);
        return s;
    }
}
```

```
// Task 7: MyMath without static - needs an object: new
MyMathInstance().Sin(x, N)
// Difference: field L belongs to the instance, so every object can have its own L
value
public class MyMathInstance {
   double L = Math.PI;

   double Exp(double x, int N) {
      double s = 0, q = 1;
      for (int i = 0; i < N; i++) { s += q; q *= x / (i+1); }
      return s + q;
   }
   double Sin(double x, int N) {
      double s = 0, q = x;
      for (int i = 0; i < N; i++) { s += q; q *= (-1) * x * x / (2*i+2) / (2*i+3); }
      return s + q;
   }
   double Cos(double x, int N) {
      double s = 0, q = 1;
      for (int i = 0; i < N; i++) { s += q; q *= (-1) * x * x / (2*i+1) / (2*i+2); }
      return s + q;
   }
   double BesselJ(double x, int N) {
      double s = 0, q = 1;
      for (int i = 0; i < N; i++) { s += q; q *= (-1) * x * x / 4 / (i+1) / (i+1); }
      return s + q;
   }
   double FourSin(double x, double[] a) {
      double s = 0;
      for (int i = 0; i < a.length; i++) s += a[i] * Math.sin(Math.PI * x * (i+1) /
L);
      return s;
   }
   double FourCos(double x, double[] a) {
      double s = 0;
      for (int i = 0; i < a.length; i++) s += a[i] * Math.cos(Math.PI * x * i / L);
      return s;
   }
```

```
}

public class Main {

    static void sep(String t) { System.out.println("\n=== " + t + " ==="); }

    public static void main(String[] args) {

        // ---- TASK 1: Vehicle ----------------------------------------------
        sep("TASK 1 - Vehicle");

        Vehicle car = new Vehicle("Oleksii", "Toyota", "Camry", 2022, "Blue");
        car.addFuel(50);
        car.addFuel(-5);          // validation
        car.setSpeed(90);
        car.setDirection(45);
        car.drive(1.0);
        car.turn(90);
        car.drive(0.5);
        System.out.println(car);
        System.out.printf("Distance from (0,0): %.2f km%n",
car.distanceFromOrigin());
        car.stop();

        Vehicle car2 = new Vehicle("", "BMW", "X5"); // empty owner
        System.out.println("Owner: " + car2.getOwner()); // Unknown
        car2.drive(1.0);          // no fuel
        car2.setSpeed(-30);       // negative speed

        // ---- TASK 2: Polynom - all methods ---------------------------------
        sep("TASK 2 - Polynom");

        Polynom P = new Polynom(new double[]{1, 2, 3}); // 1 + 2x + 3x^2
        Polynom Q = new Polynom(new double[]{2, 1});    // 2 + x

        System.out.print("P: "); P.show();
        System.out.print("Q: "); Q.show();
        System.out.print("P+Q: "); P.plus(Q).show();
        System.out.print("P-Q: "); P.minus(Q).show();
```

```
        System.out.print("P*Q: "); P.prod(Q).show();
        System.out.print("P*2: "); P.prod(2.0).show();
        System.out.print("P/2: "); P.div(2.0).show();
        System.out.print("P': ");  P.diff().show();
        System.out.print("P": "); P.diff(2).show();
        P.show(2);                              // P(2) = 17
        System.out.print("null: "); new Polynom(null).show();

        // ---- TASK 3: static methods ----------------------------------------
        sep("TASK 3 - Static methods of Polynom");
        System.out.print("add:        "); Polynom.add(P, Q).show();
        System.out.print("sub:        "); Polynom.sub(P, Q).show();
        System.out.print("multiply:   "); Polynom.multiply(P, Q).show();
        System.out.print("derivative: "); Polynom.derivative(P, 1).show();
        System.out.println("evaluate P(2): " + Polynom.evaluate(P, 2));

        // ---- TASK 4: new static methods -----------------------------------
        sep("TASK 4 - New static methods of Polynom");
        System.out.print("integrate(P): ");    Polynom.integrate(P).show();
        System.out.printf("Integral_0^1 P(x) dx = %.4f  (expected 3.0)%n",
              Polynom.definiteIntegral(P, 0, 1));
        System.out.printf("norm(P) = %.4f  (expected %.4f)%n",
Polynom.norm(P), Math.sqrt(14));
        System.out.print("fromRoots(1,-1,2): "); Polynom.fromRoots(new
double[]{1, -1, 2}).show();

        // ---- TASK 5: MyMath ----------------------------------------------
        sep("TASK 5 - MyMath (static)");
        System.out.printf("Exp(1,50)    = %.10f | reference: %.10f%n",
MyMath.Exp(1,50), Math.exp(1));
        System.out.printf("Sin(pi/4,50) = %.10f | reference: %.10f%n",
MyMath.Sin(Math.PI/4,50), Math.sin(Math.PI/4));
        System.out.printf("Cos(pi/4,50) = %.10f | reference: %.10f%n",
MyMath.Cos(Math.PI/4,50), Math.cos(Math.PI/4));
        double[] fa = {1.0, 1.0/3, 1.0/5};
        System.out.printf("FourSin(0.5) = %.6f%n", MyMath.FourSin(0.5, fa));
        System.out.printf("FourCos(0.5) = %.6f%n", MyMath.FourCos(0.5, fa));

        // ---- TASK 6: BesselJ(mu1) ~ 0 -------------------------------------
```

```
        sep("TASK 6 - BesselJ(2.404825558) ~ 0");
        double mu1 = 2.404825558;
        for (int n : new int[]{5, 10, 20, 50})
            System.out.printf("N=%2d: %+.2e  %s%n", n, MyMath.BesselJ(mu1, n),
                    Math.abs(MyMath.BesselJ(mu1, n)) < 1e-6 ? "~0 OK" : "");

        // ---- TASK 7: MyMathInstance (non-static) ---------------------------
        sep("TASK 7 - MyMathInstance (methods without static)");
        // static  -> MyMath.Sin(x, N)            - call by class name
        MyMathInstance m1 = new MyMathInstance();       // L = pi
        MyMathInstance m2 = new MyMathInstance();
        m2.L = 2 * Math.PI;                             // its own L for each object

        System.out.printf("m1.Exp(1,50)       = %.10f%n", m1.Exp(1, 50));
        System.out.printf("m1.BesselJ(mu1,50) = %+.2e (~0)%n",
m1.BesselJ(mu1, 50));
        System.out.printf("m1.FourSin(0.5) L=pi   = %.6f%n", m1.FourSin(0.5,
fa));
        System.out.printf("m2.FourSin(0.5) L=2pi  = %.6f  <- different result!%n",
m2.FourSin(0.5, fa));
    }
}
```

Приклад Роботи

```
Coefficient:    2.0     5.0     8.0     3.0
derivative: Degree of x:        0       1
Coefficient:    2.0     6.0
evaluate P(2): 17.0

=== TASK 4 - New static methods of Polynom ===
integrate(P): Degree of x:      0       1       2       3
Coefficient:    0.0     1.0     1.0     1.0
Integral_0^1 P(x) dx = 3,0000  (expected 3.0)
norm(P) = 3,7417  (expected 3,7417)
fromRoots(1,-1,2): Degree of x:  0      1       2       3
Coefficient:    2.0     -1.0    -2.0    1.0

=== TASK 5 - MyMath (static) ===
Exp(1,50)    = 2,7182818285 | reference: 2,7182818285
Sin(pi/4,50) = 0,7071067812 | reference: 0,7071067812
Cos(pi/4,50) = 0,7071067812 | reference: 0,7071067812
FourSin(0.5) = 0,959415
FourCos(0.5) = 1,400588

=== TASK 6 - BesselJ(2.404825558) ~ 0 ===
N= 5: -1,71e-05
N=10: -1,58e-10  ~0 OK
N=20: -1,58e-10  ~0 OK
N=50: -1,58e-10  ~0 OK

=== TASK 7 - MyMathInstance (methods without static) ===
m1.Exp(1,50)       = 2,7182818285
m1.BesselJ(mu1,50) = -1,58e-10 (~0)
m1.FourSin(0.5) L=pi   = 0,959415
m2.FourSin(0.5) L=2pi  = 0,543540  <- different result!
```

Контрольні Запитання

1. Поведінка об’єкту визначається його методами – тим, що він може робити.

2. Стан об’єкту визначається його полями або змінними екземпляра – даними, які він зберігає.

3. Клас задає шаблон для об’єктів; об’єкт є конкретним екземпляром класу.

4. Конструктор в класі ініціалізує об’єкт під час його створення.

5. Змінна екземпляра – це поле конкретного об’єкту; метод екземпляра – це функція, яка працює з цим об’єктом.

6. Можна створити скільки завгодно екземплярів одного класу (залежить від пам’яті).

7. “Перевантаження методів” означає наявність у класі кількох методів з однаковим іменем, але різними параметрами.

8. Перевантажені методи відрізняються кількістю або типами параметрів.

9. Конструктор після використання new повертає посилання на створений об’єкт.

10. Щоб отримати доступ до члена екземпляра зі статичного методу, потрібно спершу створити об’єкт цього класу і через нього звертатися до його членів.